

AI RESUME JOB RECOMMENDATION SYSTEM

Research Paper

Your Name: K.ROJASRI
B.Tech Information Technology
College Name
Academic Year 2026–2027

Abstract

Job seekers often spend considerable time searching job descriptions and manually comparing required skills with their resumes. This paper presents an AI-assisted Resume Job Recommendation System implemented as a Flask web application. The system accepts PDF/DOCX resumes, extracts resume text and candidate information, and compares the profile with job records. The project combines skill and keyword matching, natural-language text similarity using TF-IDF and cosine similarity, and experience compatibility to calculate recommendation scores. The result is presented through a web interface containing resume analysis, job recommendations, job details, dashboard, skill-gap and career-insight pages. The implementation demonstrates a practical approach to automated resume-job matching and provides a foundation for future extensions using larger datasets, advanced NLP models and live job sources.

Keywords:

Resume Analysis, Job Recommendation, Natural Language Processing, TF-IDF, Cosine Similarity, Skill Matching, Flask, Machine Learning

1. Introduction

Recruitment and job searching involve matching candidate profiles with job requirements. A resume contains information about skills, education, experience and career interests, while job descriptions contain required skills, role information and experience expectations. Manual comparison can be time-consuming and may not identify all relevant opportunities. An automated recommendation system can reduce this effort by transforming resume information into features and comparing it with job data.

The proposed project is a web-based AI Resume Job Recommendation System. It is implemented using Python and Flask, with data processing and NLP components. The uploaded project contains the main Flask application, HTML templates, datasets and a testing/analysis script. The application provides multiple pages for resume upload, analysis, job browsing, job details, dashboard, saved jobs, resume history, skill-gap analysis and career insights.

2. Problem Statement

Candidates may have difficulty identifying jobs that match their skills and experience because job descriptions can be numerous and information is presented in different formats. A system is required to automatically process a resume, compare its information with job requirements and provide relevant job recommendations through a simple interface.

3. Objectives

- Develop a web application for resume-based job recommendation.
- Extract text from uploaded PDF and DOCX resumes.
- Identify and compare candidate skills and job requirements.
- Use NLP similarity to compare resume text with job information.
- Consider experience compatibility while calculating recommendations.
- Display recommendations and analysis through an interactive web interface.

4. Related Work

Existing recruitment platforms commonly use keyword search, filtering and profile-based matching. Traditional keyword methods can be effective for exact skill terms but may overlook semantic similarity between a resume and a job description. NLP-based approaches represent documents numerically and can compare their similarity. TF-IDF is a classical information-retrieval technique that emphasizes terms that are important within a document relative to a collection, while cosine similarity measures the angle-based similarity between vector representations.

The present project combines direct skill/keyword matching with TF-IDF-based similarity and experience compatibility. This hybrid approach is intended to use both explicit skills and broader textual information rather than depending on only one signal.

5. Proposed System

The system accepts a candidate resume through the web interface. The uploaded document is processed to obtain readable text. Candidate information is then compared with job records stored in the project data. Skills and keywords are matched explicitly, text similarity is calculated using NLP, and experience compatibility is considered. The combined score is used to rank job recommendations.

Processing flow: User ☐ Resume Upload ☐ Text Extraction ☐ Candidate/Skill Analysis ☐ Job Dataset ☐ Skill & Keyword Matching ☐ TF-IDF/Cosine Similarity ☐ Experience Matching ☐ Weighted Recommendation Score ☐ Recommended Jobs.

6.Methodology

- a. **Resume Text Extraction:** The application supports document processing for PDF and DOCX resumes. Extracted text becomes the input for subsequent analysis.
- b. **Skill and Keyword Matching:** Candidate skills and job skills/keywords are normalized and compared. The proportion of common skills contributes to the matching score.
- c. **TF-IDF Representation:** Resume and job text are converted into TF-IDF vectors. This represents the importance of terms while reducing the effect of very common words.
- d. **Cosine Similarity:** The vector representations are compared using cosine similarity. A higher similarity indicates that the resume and job text have more similar term patterns.
- e. **Experience Compatibility:** Candidate experience is compared with the experience expected by a job. This produces an additional compatibility component.
- f. **Weighted Recommendation:** The project combines the matching components into an overall recommendation score. Jobs can then be sorted using the resulting score and displayed as recommendations.

7. Dataset

The project contains two CSV datasets: **job_dataset.csv** and **ml_resume_dataset_4500.csv**. The testing script loads these datasets and performs basic inspection including dimensions, columns, missing values and duplicate records.

Resume records contain profile-related fields and resume text, while job records provide information needed for job comparison.

The included testing logic also performs skill cleaning and evaluates skill matching and text similarity. Dataset quality and coverage directly affect the usefulness of the recommendations.

8. System Modules

Resume Upload: Receives a candidate resume and starts the analysis workflow.

Resume Analysis: Presents extracted and processed candidate information.

Job Recommendation: Ranks jobs using the matching logic.

Job Details: Shows information associated with an individual job.

Skill Gap Analysis: Provides information that can help identify missing or less-matched skills.

Dashboard: Provides a consolidated view of application information.

Saved Jobs: Provides a section for selected jobs.

Resume History: Provides a section for previous resume-related records.

Career Insights: Provides career-oriented information using the application's data.

9. Implementation

The main application file is `app.py`. It creates the Flask application, configures the web application and implements the application workflow. The project includes multiple HTML templates, including pages for the home screen, upload, analysis, jobs, job details, dashboard, profile, resume history, saved jobs, skill gap, career insights and about information.

The `test.py` file is used to inspect the datasets and demonstrate recommendation-related processing. The project uses Python data-processing and machine-learning libraries for the matching workflow. The implementation therefore combines conventional web development with document processing and NLP-based recommendation.

10. Experimental / Output Discussion

The expected output is a browser-based application in which a user uploads a resume and receives job recommendations. The recommendation process uses multiple matching signals. The interface also provides supporting pages so that the user can inspect analysis and job information.

The testing component is useful for checking dataset structure and demonstrating the matching process. For a production evaluation, the system should be tested using a labeled set of resume-job pairs and metrics such as Precision@K, Recall@K, F1-score or ranking-based measures. Such quantitative evaluation is not claimed here because the uploaded project does not provide a validated ground-truth benchmark.

11. Advantages

- Automates part of the resume-to-job comparison process.
- Combines explicit skill matching and text similarity.
- Includes experience compatibility in the recommendation logic.
- Supports PDF/DOCX resume processing.
- Provides a web interface suitable for academic demonstration and portfolio presentation.
- Can be extended with larger datasets and more advanced models.

12. Limitations

- Recommendations depend on the quality and coverage of the included job and resume data.
- TF-IDF is a classical text representation and may not capture all semantic relationships.
- The project does not establish production-level recommendation accuracy without a labeled evaluation dataset.
- The included job data is not equivalent to a continuously updated live job market.

- Production deployment would require additional security, authentication, persistent database design and secure file management.

13. Future Enhancements

- Use transformer-based embeddings or modern language models for semantic matching.
- Train and evaluate a supervised ranking/classification model using labeled resume-job pairs.
- Integrate live job APIs or a continuously maintained job database.
- Add personalized course and skill-learning recommendations.
- Add secure authentication and a production database.
- Deploy the application to a cloud environment with secure document storage and monitoring.

14. Conclusion

The AI Resume Job Recommendation System demonstrates a practical integration of Flask web development, document processing, data analysis and NLP-based recommendation. The project processes resume information and compares it with job records using skill/keyword matching, TF-IDF similarity and experience compatibility. The resulting web interface provides job recommendations together with analysis and career-oriented modules. The system forms a useful academic prototype and can be extended through modern NLP models, larger datasets, quantitative evaluation and live job data.

15. References

1. Scikit-learn documentation — TF-IDF vectorization and cosine similarity.
2. Flask documentation — Python web application framework.
3. Pandas documentation — data analysis and CSV processing.
4. Python documentation — programming language and standard libraries.
5. pypdf and python-docx documentation — document text extraction.